

Exploits de kernel (ultimo recurso)

Objetivos de aprendizaje

Identificar la superficie, explicar la causa, diseñar una confirmación mínima, interpretar evidencia y proponer una mitigación causal.

Prerrequisitos

Flujo source-transformación-validación-sink, parsers, permisos y metodología de hipótesis. Revisa los prerrequisitos declarados en los metadatos.

Fundamentos técnicos

Un exploit de kernel aprovecha un fallo en el propio nucleo de Linux para ejecutar codigo como root desde un usuario normal. Casos famosos: DirtyCow, PwnKit (polkit), DirtyPipe. Funcionan cuando la version del kernel o de un componente del sistema es vulnerable, independientemente de la config.

Se dejan para el FINAL por dos razones: los exploits de kernel pueden causar un kernel panic y tumbar la maquina (arruinando la room), y muchas veces hay un vector de config mas limpio y estable. Solo cuando la enumeracion (enum-privesc-linux) no dio nada mejor, se recurre a esto.

Modelo mental

Entrada controlada -> transformaciones -> validación -> componente final
-> sink -> efecto observable -> decisión

El paso crítico es demostrar qué componente consume el valor y con qué identidad o privilegios.

Superficie de ataque

Cuando ningun vector de config (sudo, SUID, cron, capabilities) funciono y uname -r revela un kernel viejo o hay un componente vulnerable (polkit, sudo Baron Samedit).

Cómo identificarla

- uname -r muestra un kernel antiguo con CVE conocido.
- linpeas sugiere exploits de kernel aplicables.
- Componentes vulnerables presentes (pkexec para PwnKit, sudo < 1.9.5p2 para Baron Samedit).

Preguntas que debo hacerme

- ¿Qué dato controlo exactamente y en qué formato viaja?
- ¿Qué transformaciones y normalizaciones ocurren antes del sink?
- ¿Qué identidad ejecuta la operación y qué permisos tiene?
- ¿Qué otra explicación produciría la misma señal?
- ¿Cuál es la prueba de menor riesgo que separa ambas explicaciones?

Prueba mínima

Compara uname -a con CVEs conocidos (DirtyCow, PwnKit, DirtyPipe) para ese rango de version.

Evidencia esperada: Tras ejecutar el exploit compilado, un prompt root y uid=0. Si no aplica, la maquina puede quedar inestable: usalo con cuidado.

Construcción progresiva del payload

1. Reproduce una entrada válida.
2. Aísla un único valor controlable.

3. Define la primitiva mínima indicada por la fuente.
4. Predice resultado y control negativo.
5. Aplica una sola adaptación cuando haya evidencia de restricción.
6. Confirma de forma reproducible antes de ampliar impacto.

Capa 1: prueba documentada

```
uname -a; cat /etc/os-release
```

Objetivo y contexto: La version exacta del kernel y de la distro es lo que decide que exploit aplica. Un exploit para otra version puede no funcionar o tumbar la maquina.

Resultado esperado: Version del kernel (ej. 4.4.0) y distro. Cruzalo con searchsploit/CVEs; kernels < ~4.8 huelen a DirtyCow, cualquiera con pkexec vulnerable a PwnKit.

Capa 2: prueba documentada

```
searchsploit linux kernel <version>
```

Objetivo y contexto: Busca exploits locales publicos para esa version concreta de kernel. Prioriza los locales (Local Privilege Escalation) sobre los remotos.

Resultado esperado: Titulos con su rango de version. Lee el PoC antes de lanzarlo (exploitar-cve); comprueba que la version encaja para no arriesgar un panic.

Anatomía de los payloads

La primera prueba es `uname -a; cat /etc/os-release`. Separa sus delimitadores, operadores, opciones, operandos y variables; algunos elementos no estarán presentes según el contexto. Las comillas, barras y separadores pertenecen a una capa concreta. Usa la explicación de cada capa para determinar qué símbolo altera sintaxis y cuál transporta datos.

Variaciones según el contexto

No traslades una prueba entre sistemas operativos, motores, frameworks o versiones sin revisar sintaxis y comportamiento. Conserva la primitiva y vuelve a serializarla para el parser real.

Filtros y bypasses

No existe un bypass universal. Registra la forma enviada, la forma decodificada, la comparación, la normalización y la forma consumida. Una adaptación es válida solo si demuestra una discrepancia concreta; si la validación ocurre sobre la forma canónica y expresa la propiedad correcta, la variante no debe funcionar.

Evidencias de confirmación

Tras ejecutar el exploit compilado, un prompt root y uid=0. Si no aplica, la maquina puede quedar inestable: usalo con cuidado.

Usa además una entrada inocua y un control negativo. No confundas reflexión, error genérico o demora aislada con confirmación.

Escalado de impacto

- Saca la version exacta con `uname -a` y las versiones de sudo/polkit.
- Busca un exploit para esa version y leelo antes de ejecutarlo (exploitar-cve).
- Compila en una maquina compatible (o en la victima si hay gcc) y transfierelo.
- Ejecuta con cuidado; confirma uid=0. Si falla o cuelga, vuelve a los vectores de config.

Amplía únicamente dentro del laboratorio y cuando cada salto aporte una capacidad nueva demostrable.

Errores frecuentes

- El kernel/distro esta parcheado contra los CVEs conocidos para esa version.

- Quedan vectores de configuración sin probar (sudo, SUID, cron): prueba esos primero.
- Ejecutar la prueba sin adaptar variables ni versión.
- Cambiar varias capas a la vez.
- Omitir el control negativo o no guardar evidencia.

Diagnóstico de payloads fallidos

Síntoma	Posible causa	Prueba de diagnóstico	Adaptación
Rechazo inmediato	Formato o precondition	Repetir entrada válida	Corregir transporte
Sin diferencia	Entrada ignorada o canal ciego	Marcador y control negativo	Buscar evidencia adecuada
Error del componente	Contexto o versión	Reducir a prueba mínima	Consultar manual detectado
Resultado parcial	Permisos/restricción	Comprobar identidad y alcance	Reducir primitiva

Mitigaciones

Eliminar el dato controlable del sink cuando sea posible; usar APIs estructuradas, allowlists sobre valores canónicos, privilegio mínimo, autorización en servidor y registros que permitan detectar abuso. La defensa concreta debe impedir la causa explicada en Fundamentos técnicos.

Relación con pentesting y certificaciones

Se espera reconocer la señal, justificar la prueba elegida, adaptar variables, interpretar salida y documentar impacto y mitigación. La puntuación debe premiar razonamiento y evidencia, no memoria literal.

Caso guiado

Parte de una señal de la lista anterior. Escribe observación e hipótesis, ejecuta la prueba mínima, compara con el control y clasifica el resultado como no confirmado, indicio o confirmación. Solo entonces sigue los pasos de escalado relevantes.

Caso de adaptación

Si la prueba básica falla, no cambies caracteres al azar. Comprueba primero transporte, parser, versión, permisos y canal de evidencia. Diseña una segunda prueba que discrimine entre las dos causas más probables.

Ejercicios

1. Señala source, transformaciones y sink en el caso guiado.
2. Explica qué evidencia refutaría la hipótesis.
3. Descompón la primera prueba documentada por opciones y argumentos.
4. Propón un control negativo y una mitigación causal.

Resumen

Si el kernel es viejo, un exploit local eleva a root; deja esto para el final porque puede tumbar la máquina. La técnica queda confirmada solo cuando una prueba mínima produce la evidencia prevista y descarta explicaciones alternativas.

Chuleta operativa

1. Confirmar alcance y condiciones.
2. Identificar entrada, componente y permisos.

3. Ejecutar prueba mínima y control.
4. Interpretar señal antes de escalar.
5. Guardar evidencia y mitigación.

Referencias

- [Concepto fuente de THM Fieldbook \(kernel-exploits-linux\)](#)
- [Referencia técnica externa](#)

Navegación

Anterior: [Construcción de payloads](#). Índice: [curso](#). Ejercicios y chuleta se enlazarán desde la matriz de trazabilidad.